NAME : MANSHI VERMA ROLL NO: 30 CUID: CU24250193 LABSHEET 4 COMPUTER VISION

1. Build a tool that computes and visualizes the 2d fourier spectrum(maagnitude and phase) of an image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image in grayscale
image = cv2.imread("/content/images.jpg", cv2.IMREAD_GRAYSCALE)

# Check if image was loaded successfully
if image is None:
    print("Error: Image not found!")
    exit()

# Compute the 2D Fourier Transform
f_transform = np.fft.fft2(image)

# Shift the zero-frequency component to the center
f_shift = np.fft.fftshift(f_transform)

# Calculate magnitude spectrum
magnitude_spectrum = 20 * np.log(np.abs(f_shift) + 1)

# Calculate phase spectrum
phase_spectrum = np.angle(f_shift)

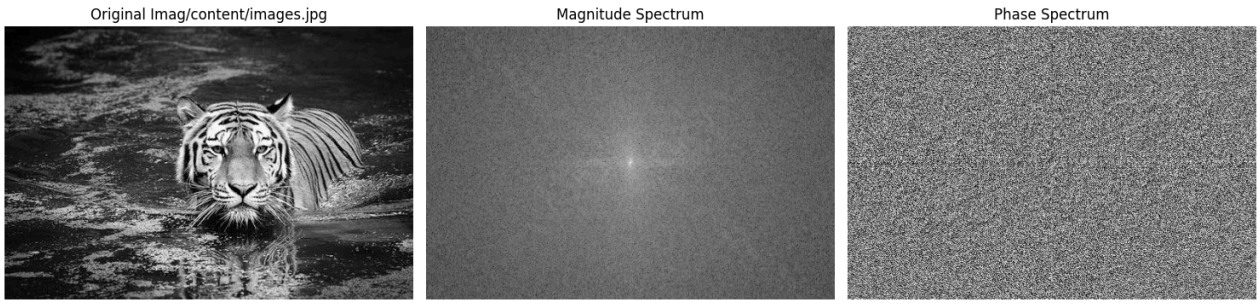
# Display original image, magnitude spectrum and phase spectrum
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Imag/content/images.jpg")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(magnitude_spectrum, cmap='gray')
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(phase_spectrum, cmap='gray')
plt.title("Phase Spectrum")
plt.axis("off")

plt.tight_layout()
plt.show()
```



2. Develop a frquency domain low pass filter (ideal,butterworth,and gaussian) and compare the resulting blur effects.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# 1. Read image in grayscale
# ------------------------------------------------------------
image = cv2.imread("/content/images (2).jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: image.jpg not found!")
    exit()

# ------------------------------------------------------------
# 2. Fourier Transform
# ------------------------------------------------------------
rows, cols = image.shape
crow, ccol = rows // 2, cols // 2

# Fourier transform
f = np.fft.fft2(image)
fshift = np.fft.fftshift(f)

# Magnitude spectrum
magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)

# ------------------------------------------------------------
# 3. Create frequency coordinate grid
# ------------------------------------------------------------
u = np.arange(cols) - ccol
v = np.arange(rows) - crow
```

```
U, V = np.meshgrid(u, v)

D = np.sqrt(U**2 + V**2)

# Cutoff frequency
D0 = 50

# ----------------------------------------------------------
# 4. Ideal Low Pass Filter
# ----------------------------------------------------------
ideal_filter = np.zeros((rows, cols), np.float32)
ideal_filter[D <= D0] = 1

# ----------------------------------------------------------
# 5. Butterworth Low Pass Filter
# ----------------------------------------------------------
n = 2  # Filter order

butterworth_filter = 1 / (1 + (D / D0) ** (2 * n))

# ----------------------------------------------------------
# 6. Gaussian Low Pass Filter
# ----------------------------------------------------------
gaussian_filter = np.exp(-(D ** 2) / (2 * (D0 ** 2)))

# ----------------------------------------------------------
# 7. Apply filters
# ----------------------------------------------------------
ideal_result = fshift * ideal_filter
butterworth_result = fshift * butterworth_filter
gaussian_result = fshift * gaussian_filter

# ----------------------------------------------------------
# 8. Inverse Fourier Transform
# ----------------------------------------------------------
ideal_result = np.fft.ifftshift(ideal_result)
ideal_result = np.fft.ifft2(ideal_result)
ideal_result = np.abs(ideal_result)

butterworth_result = np.fft.ifftshift(butterworth_result)
butterworth_result = np.fft.ifft2(butterworth_result)
butterworth_result = np.abs(butterworth_result)

gaussian_result = np.fft.ifftshift(gaussian_result)
gaussian_result = np.fft.ifft2(gaussian_result)
gaussian_result = np.abs(gaussian_result)

# Normalize results
ideal_result = cv2.normalize(
    ideal_result, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

butterworth_result = cv2.normalize(
    butterworth_result, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

gaussian_result = cv2.normalize(
    gaussian_result, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

# ----------------------------------------------------------
# 9. Display Results
# ----------------------------------------------------------
plt.figure(figsize=(14, 10))

plt.subplot(2, 4, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 4, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")

plt.subplot(2, 4, 3)
plt.imshow(ideal_filter, cmap="gray")
plt.title("Ideal LPF")
plt.axis("off")

plt.subplot(2, 4, 4)
plt.imshow(ideal_result, cmap="gray")
plt.title("Ideal LPF Result")
plt.axis("off")

plt.subplot(2, 4, 5)
plt.imshow(butterworth_filter, cmap="gray")
plt.title("Butterworth LPF")
plt.axis("off")

plt.subplot(2, 4, 6)
plt.imshow(butterworth_result, cmap="gray")
plt.title("Butterworth Result")
plt.axis("off")

plt.subplot(2, 4, 7)
plt.imshow(gaussian_filter, cmap="gray")
plt.title("Gaussian LPF")
plt.axis("off")

plt.subplot(2, 4, 8)
plt.imshow(gaussian_result, cmap="gray")
plt.title("Gaussian Result")
plt.axis("off")

plt.tight_layout()
plt.show()

# ----------------------------------------------------------
# 10. Separate comparison
# ----------------------------------------------------------
plt.figure(figsize=(15, 5))
```

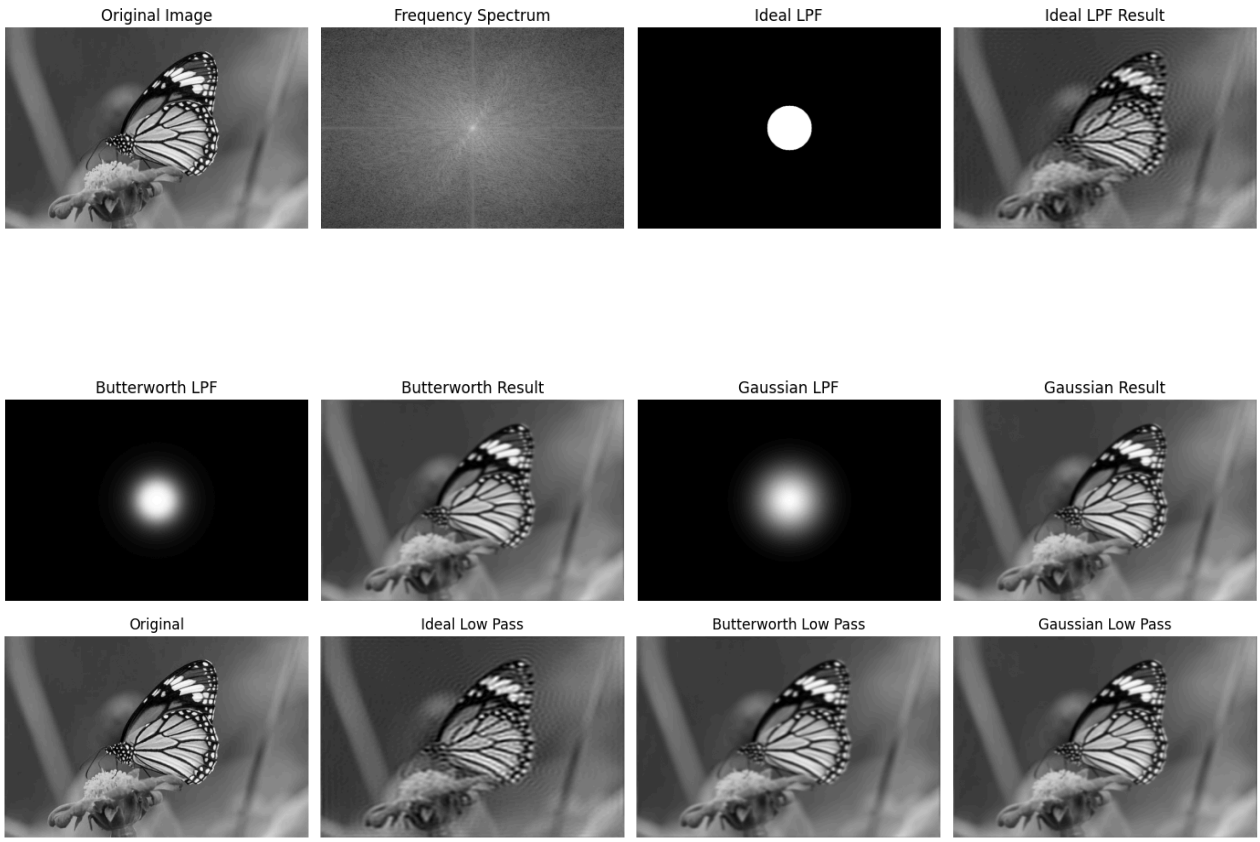
```
plt.subplot(1, 4, 1)
plt.imshow(image, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(ideal_result, cmap="gray")
plt.title("Ideal Low Pass")
plt.axis("off")

plt.subplot(1, 4, 3)
plt.imshow(butterworth_result, cmap="gray")
plt.title("Butterworth Low Pass")
plt.axis("off")

plt.subplot(1, 4, 4)
plt.imshow(gaussian_result, cmap="gray")
plt.title("Gaussian Low Pass")
plt.axis("off")

plt.tight_layout()
plt.show()
```



3. Create a frequency domain high pass filter to enhance edges , and comparwe the result with spatial domain edge detction.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# 1. Read the image
# ------------------------------------------------------------
image = cv2.imread("/content/images (3).jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: image.jpg not found!")
    print("Please keep image.jpg in the same folder as this code.")
else:

    # ------------------------------------------------------------
    # 2. Frequency Domain High-Pass Filter
    # ------------------------------------------------------------

    # Fourier Transform
    f = np.fft.fft2(image)

    # Shift zero frequency to center
    fshift = np.fft.fftshift(f)

    # Magnitude spectrum
    magnitude_spectrum = 20 * np.log(np.abs(fshift) + 1)

    # Image dimensions
    rows, cols = image.shape
    crow, ccol = rows // 2, cols // 2

    # Create High-Pass Filter
    # Low frequencies near the center are removed
```

```
radius = 30

y, x = np.ogrid[:rows, :cols]
distance = np.sqrt((x - ccol) ** 2 + (y - crow) ** 2)

high_pass_filter = np.ones((rows, cols), np.uint8)
high_pass_filter[distance <= radius] = 0

# Apply High-Pass Filter
filtered = fshift * high_pass_filter

# Inverse Fourier Transform
filtered_shift = np.fft.ifftshift(filtered)
high_pass_result = np.fft.ifft2(filtered_shift)

# Magnitude of result
high_pass_result = np.abs(high_pass_result)

# Normalize result
high_pass_result = cv2.normalize(
    high_pass_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# ------------------------------------------------------------
# 3. Spatial Domain Edge Detection
# ------------------------------------------------------------

# Sobel Edge Detection
sobel_x = cv2.Sobel(
    image,
    cv2.CV_64F,
    1,
    0,
    ksize=3
)

sobel_y = cv2.Sobel(
    image,
    cv2.CV_64F,
    0,
    1,
    ksize=3
)

sobel = cv2.magnitude(sobel_x, sobel_y)
sobel = cv2.normalize(
    sobel,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# Canny Edge Detection
canny = cv2.Canny(image, 100, 200)

# ------------------------------------------------------------
# 4. Display Results
# ------------------------------------------------------------

plt.figure(figsize=(16, 10))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Fourier Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(high_pass_filter, cmap="gray")
plt.title("High-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(high_pass_result, cmap="gray")
plt.title("Frequency Domain HPF")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(sobel, cmap="gray")
plt.title("Spatial Domain - Sobel")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(canny, cmap="gray")
plt.title("Spatial Domain - Canny")
plt.axis("off")

plt.tight_layout()
plt.show()

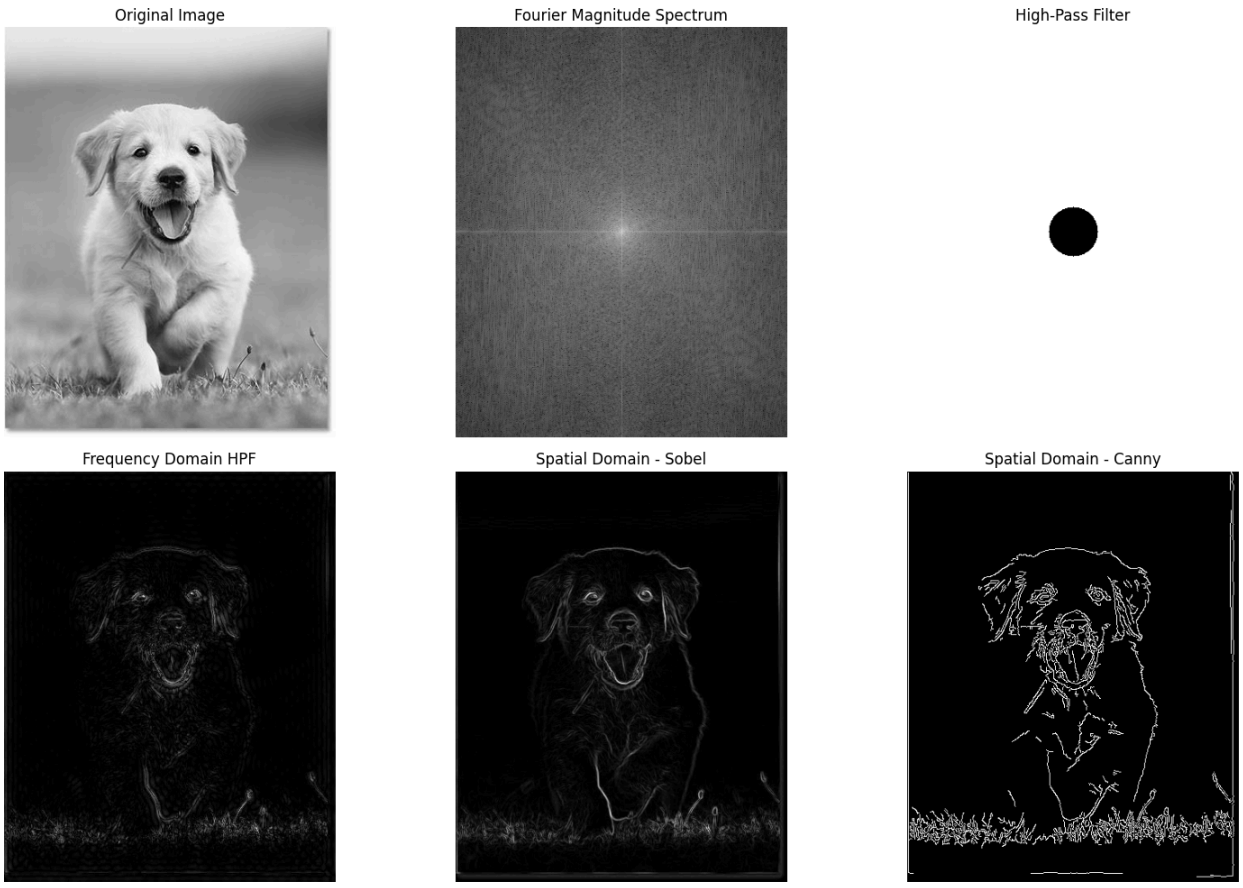
# ------------------------------------------------------------
# 5. Comparison
# ------------------------------------------------------------

print("\n========== COMPARISON ==========")
print("Frequency Domain High-Pass Filter:")
print("- Enhances high-frequency components.")
print("- Highlights edges and fine image details.")
print("- Can also enhance noise.")

print("\nSpatial Domain Sobel:")
print("- Detects horizontal and vertical intensity changes.")
print("- Simple and computationally efficient.")
```

```
    print("- Produces gradient-based edge information.")

    print("\nSpatial Domain Canny:")
    print("- Produces thin and well-defined edges.")
    print("- Uses smoothing, gradient calculation and thresholding.")
    print("- Generally gives cleaner edge maps.")
```




```
========== COMPARISON ==========
Frequency Domain High-Pass Filter:
- Enhances high-frequency components.
- Highlights edges and fine image details.
- Can also enhance noise.

Spatial Domain Sobel:
- Detects horizontal and vertical intensity changes.
- Simple and computationally efficient.
- Produces gradient-based edge information.

Spatial Domain Canny:
- Produces thin and well-defined edges.
- Uses smoothing, gradient calculation and thresholding.
- Generally gives cleaner edge maps.
```

4. Build a periodic noise removal tool that identifies and removes noise spikes visible in the frequency spectrum.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. Read Image
# ============================================================

image = cv2.imread("/content/images (4).jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: image.jpg not found!")
else:

    # ========================================================
    # 2. Add Periodic Noise
    #    (This makes the experiment visible even if the
    #     original image does not contain periodic noise.)
    # ========================================================

    rows, cols = image.shape

    y, x = np.indices((rows, cols))

    periodic_noise = (
        25 * np.sin(2 * np.pi * x / 12) +
        25 * np.sin(2 * np.pi * y / 18)
```

```
)

noisy_image = image.astype(np.float32) + periodic_noise
noisy_image = np.clip(noisy_image, 0, 255).astype(np.uint8)

# ============================================================
# 3. Fourier Transform
# ============================================================

f = np.fft.fft2(noisy_image)

# Move zero frequency to center
fshift = np.fft.fftshift(f)

# Frequency spectrum
magnitude = 20 * np.log(np.abs(fshift) + 1)

# ============================================================
# 4. Detect Noise Spikes
# ============================================================

magnitude_norm = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

# Ignore the large central low-frequency region
center_radius = 20

cy, cx = rows // 2, cols // 2

Y, X = np.ogrid[:rows, :cols]
distance = np.sqrt((X - cx)**2 + (Y - cy)**2)

spectrum_for_detection = magnitude.copy()

spectrum_for_detection[distance < center_radius] = 0

# Threshold for strong frequency spikes
threshold = np.percentile(
    spectrum_for_detection[spectrum_for_detection > 0],
    99.7
)

spike_mask = spectrum_for_detection > threshold

# ============================================================
# 5. Remove Very Small/Connected Noise Points
# ============================================================

spike_mask = spike_mask.astype(np.uint8)

# Dilate slightly so that the notch filter covers the spike
kernel = np.ones((5, 5), np.uint8)
spike_mask = cv2.dilate(spike_mask, kernel, iterations=1)

# ============================================================
# 6. Create Notch-Reject Filter
# ============================================================

notch_filter = np.ones((rows, cols), np.float32)

notch_filter[spike_mask == 1] = 0

# ============================================================
# 7. Apply Notch Filter
# ============================================================

filtered_frequency = fshift * notch_filter

# Shift back
filtered_frequency = np.fft.ifftshift(filtered_frequency)

# Inverse Fourier Transform
restored = np.fft.ifft2(filtered_frequency)

restored = np.abs(restored)

# Normalize
restored = cv2.normalize(
    restored,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# ============================================================
# 8. Display Results
# ============================================================

plt.figure(figsize=(16, 10))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(noisy_image, cmap="gray")
plt.title("Image with Periodic Noise")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(magnitude, cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")

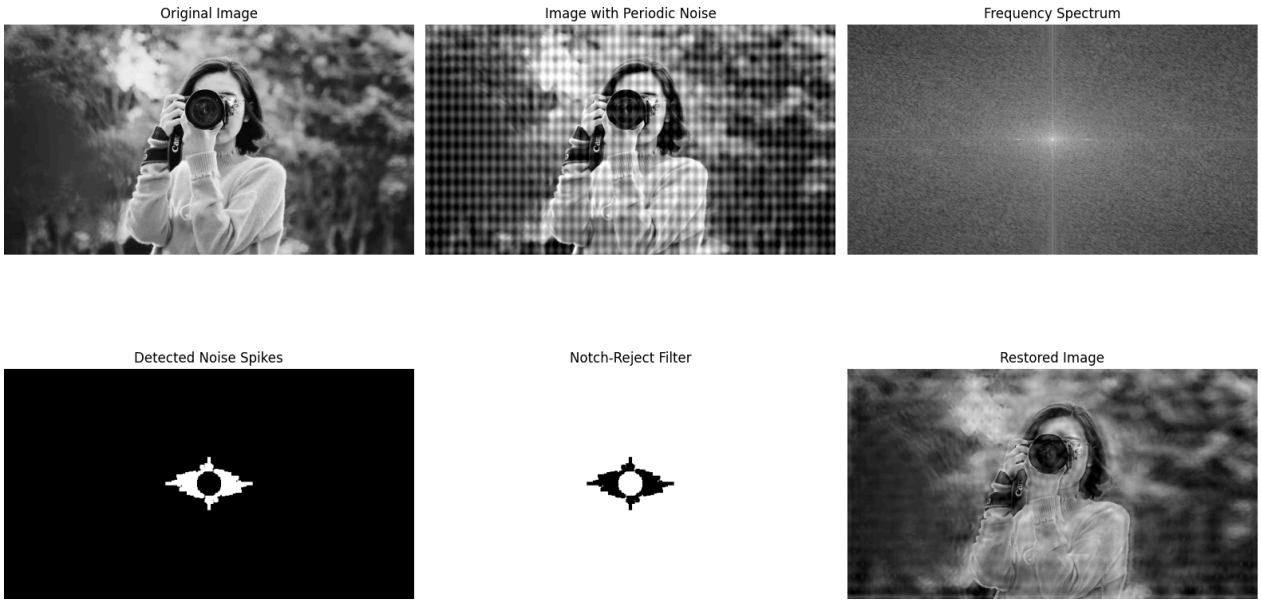
plt.subplot(2, 3, 4)
```

```
    plt.imshow(spike_mask, cmap="gray")
    plt.title("Detected Noise Spikes")
    plt.axis("off")

    plt.subplot(2, 3, 5)
    plt.imshow(notch_filter, cmap="gray")
    plt.title("Notch-Reject Filter")
    plt.axis("off")

    plt.subplot(2, 3, 6)
    plt.imshow(restored, cmap="gray")
    plt.title("Restored Image")
    plt.axis("off")

    plt.tight_layout()
    plt.show()
    # 9. Print Information
    print("========================================")
    print(" PERIODIC NOISE REMOVAL")
    print("========================================")
    print("Image size:", image.shape)
    print("Detected frequency spike pixels:",
          np.sum(spike_mask))
     print("\nMethod:")
    print("1. Fourier transform is applied.")
    print("2. Frequency spectrum is examined.")
    print("3. Strong spikes are detected.")
    print("4. Notch-reject filter removes the spikes.")
    print("5. Inverse Fourier transform reconstructs the image.")
```




```
========================================
 PERIODIC NOISE REMOVAL
========================================
Image size: (335, 597)
Detected frequency spike pixels: 2782

Method:
1. Fourier transform is applied.
2. Frequency spectrum is examined.
3. Strong spikes are detected.
4. Notch-reject filter removes the spikes.
5. Inverse Fourier transform reconstructs the image.
```

5. Develop a program that compresses an image by retaining only the top-k fourier coefficients and reconstructs it.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. Read Image
# ============================================================

image = cv2.imread("/content/images.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: image.jpg not found!")
else:

    # Convert image to float
    image_float = image.astype(np.float32)

    # ============================================================
    # 2. Fourier Transform
    # ============================================================

    F = np.fft.fft2(image_float)

    # Shift zero frequency to the center
    F_shift = np.fft.fftshift(F)
```

```
# ============================================================
# 3. Magnitude of Fourier Coefficients
# ============================================================

magnitude = np.abs(F_shift)

# ============================================================
# 4. Select Top-K Fourier Coefficients
# ============================================================

# Total number of coefficients
total_coefficients = magnitude.size

# Keep top 5% coefficients
percentage = 5

k = int(total_coefficients * percentage / 100)

# Flatten magnitude
flat_magnitude = magnitude.flatten()

# Find threshold for top-K coefficients
threshold = np.partition(
    flat_magnitude,
    -k
)[-k]

# Create mask
mask = magnitude >= threshold

# Keep only top-K coefficients
F_compressed = F_shift * mask

# ============================================================
# 5. Inverse Fourier Transform
# ============================================================

# Shift back
F_inverse_shift = np.fft.ifftshift(F_compressed)

# Reconstruct image
reconstructed = np.fft.ifft2(F_inverse_shift)

# Take magnitude
reconstructed = np.abs(reconstructed)

# Normalize to 0-255
reconstructed = cv2.normalize(
    reconstructed,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# ============================================================
# 6. Calculate Compression Ratio
# ============================================================

retained_coefficients = np.sum(mask)

compression_ratio = (
    total_coefficients / retained_coefficients
)

percentage_retained = (
    retained_coefficients / total_coefficients
) * 100

# ============================================================
# 7. Display Results
# ============================================================

plt.figure(figsize=(16, 10))

# Original image
plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Fourier spectrum
plt.subplot(2, 3, 2)
spectrum = np.log(1 + np.abs(F_shift))
plt.imshow(spectrum, cmap="gray")
plt.title("Fourier Spectrum")
plt.axis("off")

# Top-K mask
plt.subplot(2, 3, 3)
plt.imshow(mask, cmap="gray")
plt.title("Top-K Coefficients")
plt.axis("off")

# Compressed spectrum
plt.subplot(2, 3, 4)
compressed_spectrum = np.log(
    1 + np.abs(F_compressed)
)
plt.imshow(compressed_spectrum, cmap="gray")
plt.title("Compressed Fourier Spectrum")
plt.axis("off")

# Reconstructed image
plt.subplot(2, 3, 5)
plt.imshow(reconstructed, cmap="gray")
plt.title("Reconstructed Image")
plt.axis("off")

# Difference
difference = cv2.absdiff(image, reconstructed)
```

```
plt.subplot(2, 3, 6)
plt.imshow(difference, cmap="gray")
plt.title("Reconstruction Difference")
plt.axis("off")

plt.tight_layout()
plt.show()

# ============================================================
# 8. Display Information
# ============================================================

print("==========================================")
print("     FOURIER IMAGE COMPRESSION")
print("==========================================")

print("Image Size:",
      image.shape)

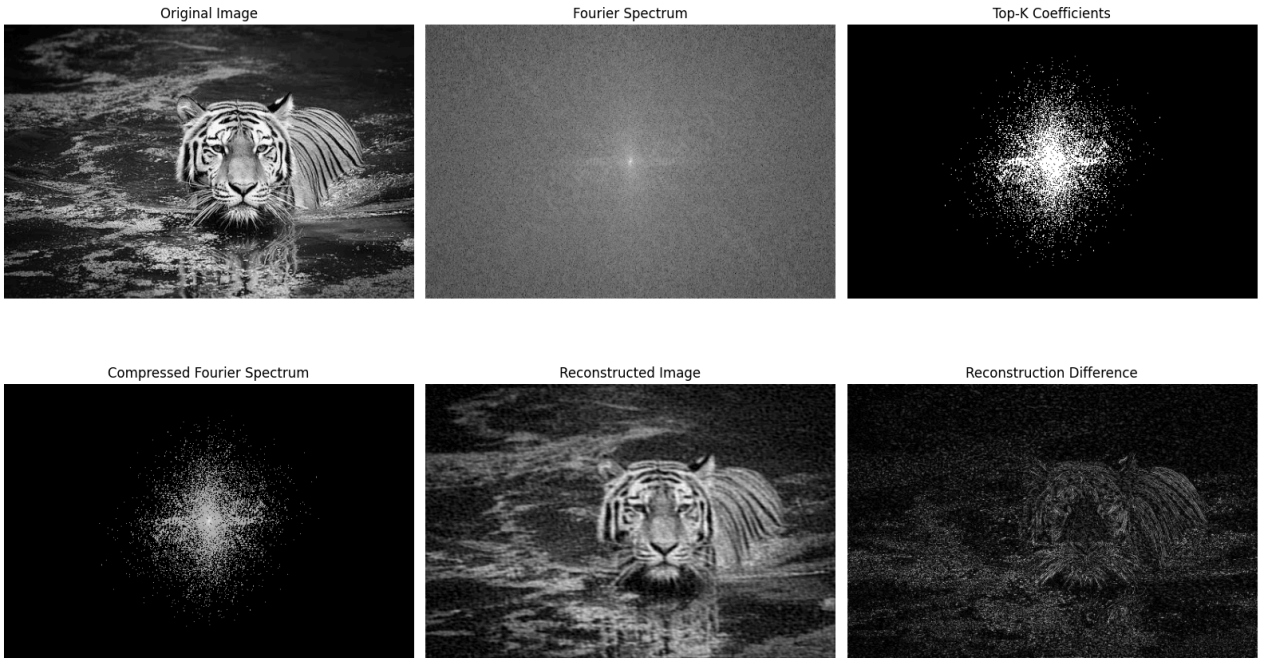
print("Total Fourier Coefficients:",
      total_coefficients)

print("Selected Top-K Coefficients:",
      retained_coefficients)

print("Percentage of Coefficients Retained:",
      round(percentage_retained, 2), "%")

print("Compression Ratio:",
      round(compression_ratio, 2), ": 1")

print("==========================================")
```




```
==========================================
     FOURIER IMAGE COMPRESSION
==========================================
Image Size: (365, 547)
Total Fourier Coefficients: 199655
Selected Top-K Coefficients: 9983
Percentage of Coefficients Retained: 5.0 %
Compression Ratio: 20.0 : 1
==========================================
```

6. Create a watermark hide/reveal tool that embeds and extract a watermark using frquency domain manipulation.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt


# ============================================================
# 1. READ IMAGES
# ============================================================

host = cv2.imread("/content/images (5).jpg", cv2.IMREAD_GRAYSCALE)
watermark = cv2.imread("/content/images (5).jpg", cv2.IMREAD_GRAYSCALE)

if host is None:
    print("Error: image.jpg not found!")
    exit()

if watermark is None:
    print("Error: watermark.png not found!")
    exit()


# ============================================================
# 2. RESIZE WATERMARK
```

```
# ============================================================

# Resize watermark to the same size as host image
watermark = cv2.resize(
    watermark,
    (host.shape[1], host.shape[0])
)

# Normalize watermark
watermark_norm = watermark.astype(np.float32) / 255.0


# ============================================================
# 3. FOURIER TRANSFORM OF HOST IMAGE
# ============================================================

host_float = np.float32(host)

# Apply FFT
dft = np.fft.fft2(host_float)

# Shift zero frequency to center
dft_shift = np.fft.fftshift(dft)


# ============================================================
# 4. CREATE WATERMARK IN FREQUENCY DOMAIN
# ============================================================

# Convert watermark into frequency-like signal
wm_frequency = watermark_norm * np.abs(dft_shift).mean()

# Embedding strength
alpha = 0.08

# Embed watermark into magnitude
magnitude = np.abs(dft_shift)
phase = np.angle(dft_shift)

watermarked_magnitude = magnitude + alpha * wm_frequency


# ============================================================
# 5. RECONSTRUCT WATERMARKED IMAGE
# ============================================================

watermarked_dft = (
    watermarked_magnitude *
    np.exp(1j * phase)
)

# Inverse shift
watermarked_dft = np.fft.ifftshift(watermarked_dft)

# Inverse FFT
watermarked_image = np.fft.ifft2(watermarked_dft)

# Take real part
watermarked_image = np.real(watermarked_image)

# Normalize to 0-255
watermarked_image = cv2.normalize(
    watermarked_image,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)


# ============================================================
# 6. SAVE WATERMARKED IMAGE
# ============================================================

cv2.imwrite(
    "watermarked_image.jpg",
    watermarked_image
)


# ============================================================
# 7. REVEAL / EXTRACT WATERMARK
# ============================================================

# FFT of watermarked image
watermarked_float = np.float32(watermarked_image)

dft_w = np.fft.fft2(watermarked_float)

dft_w_shift = np.fft.fftshift(dft_w)

# Calculate magnitude
watermarked_magnitude = np.abs(dft_w_shift)

# Original magnitude
original_magnitude = np.abs(dft_shift)

# Extract embedded information
extracted = (
    watermarked_magnitude - original_magnitude
)

# Remove negative values
extracted = np.maximum(extracted, 0)

# Normalize extracted watermark
extracted_watermark = cv2.normalize(
    extracted,
    None,
    0,
    255,
    cv2.NORM_MINMAX
```

```
).astype(np.uint8)


# ============================================================
# 8. THRESHOLD EXTRACTED WATERMARK
# ============================================================

# Improve visibility
_, extracted_binary = cv2.threshold(
    extracted_watermark,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)


# ============================================================
# 9. FREQUENCY SPECTRUM
# ============================================================

original_spectrum = 20 * np.log(
    np.abs(dft_shift) + 1
)

watermarked_spectrum = 20 * np.log(
    np.abs(dft_w_shift) + 1
)


# ============================================================
# 10. DISPLAY RESULTS
# ============================================================

plt.figure(figsize=(15, 10))


plt.subplot(2, 3, 1)
plt.imshow(host, cmap="gray")
plt.title("Original Image")
plt.axis("off")


plt.subplot(2, 3, 2)
plt.imshow(watermark, cmap="gray")
plt.title("Original Watermark")
plt.axis("off")


plt.subplot(2, 3, 3)
plt.imshow(original_spectrum, cmap="gray")
plt.title("Original Frequency Spectrum")
plt.axis("off")


plt.subplot(2, 3, 4)
plt.imshow(watermarked_image, cmap="gray")
plt.title("Watermarked Image")
plt.axis("off")


plt.subplot(2, 3, 5)
plt.imshow(watermarked_spectrum, cmap="gray")
plt.title("Watermarked Frequency Spectrum")
plt.axis("off")


plt.subplot(2, 3, 6)
plt.imshow(extracted_binary, cmap="gray")
plt.title("Extracted / Revealed Watermark")
plt.axis("off")


plt.tight_layout()
plt.show()


# ============================================================
# 11. INFORMATION
# ============================================================

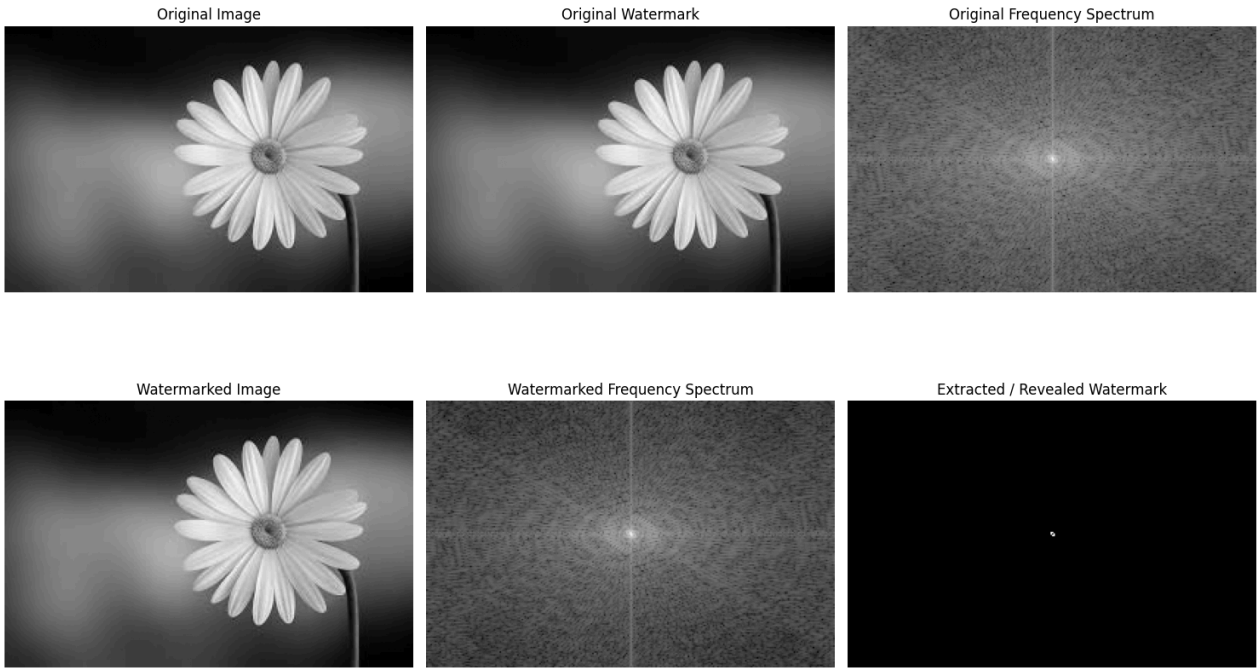
print("=" * 60)
print("FREQUENCY DOMAIN WATERMARKING")
print("=" * 60)

print("Original image       :", host.shape)
print("Watermark            :", watermark.shape)
print("Embedding strength   :", alpha)

print("\nWatermark embedding completed.")
print("Watermarked image saved as: watermarked_image.jpg")

print("\nWatermark extraction completed.")

print("=" * 60)
```




```
============================================================
FREQUENCY DOMAIN WATERMARKING
============================================================
Original image       : (181, 278)
Watermark            : (181, 278)
Embedding strength   : 0.08

Watermark embedding completed.
Watermarked image saved as: watermarked_image.jpg

Watermark extraction completed.
============================================================
```

7. Build a band-pass /band reject filter designer for isolating specific frequency ranges in a textured image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. READ TEXTURED IMAGE
# ============================================================

image = cv2.imread("/content/images (5).jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: image.jpg not found!")
    exit()

# ============================================================
# 2. FOURIER TRANSFORM
# ============================================================

dft = np.fft.fft2(image)
dft_shift = np.fft.fftshift(dft)

# Magnitude spectrum
magnitude = np.abs(dft_shift)
spectrum = 20 * np.log(magnitude + 1)

# ============================================================
# 3. CREATE DISTANCE MATRIX
# ============================================================

rows, cols = image.shape

crow = rows // 2
ccol = cols // 2

Y, X = np.ogrid[:rows, :cols]

distance = np.sqrt(
    (X - ccol) ** 2 +
    (Y - crow) ** 2
)

# ============================================================
# 4. FILTER PARAMETERS
# ============================================================

# Change these values to select different frequency ranges
low_cutoff = 20
high_cutoff = 80

# ============================================================
# 5. BAND-PASS FILTER
```

```
# ============================================================

band_pass = np.zeros((rows, cols), dtype=np.float32)

band_pass[
    (distance >= low_cutoff) &
    (distance <= high_cutoff)
] = 1

# Apply Band-Pass filter
band_pass_dft = dft_shift * band_pass

# Inverse Fourier Transform
band_pass_inverse = np.fft.ifftshift(band_pass_dft)

band_pass_result = np.fft.ifft2(
    band_pass_inverse
)

band_pass_result = np.abs(band_pass_result)

# Normalize result
band_pass_result = cv2.normalize(
    band_pass_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# ============================================================
# 6. BAND-REJECT FILTER
# ============================================================

band_reject = np.ones(
    (rows, cols),
    dtype=np.float32
)

band_reject[
    (distance >= low_cutoff) &
    (distance <= high_cutoff)
] = 0

# Apply Band-Reject filter
band_reject_dft = dft_shift * band_reject

# Inverse Fourier Transform
band_reject_inverse = np.fft.ifftshift(
    band_reject_dft
)

band_reject_result = np.fft.ifft2(
    band_reject_inverse
)

band_reject_result = np.abs(
    band_reject_result
)

# Normalize result
band_reject_result = cv2.normalize(
    band_reject_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# ============================================================
# 7. SAVE RESULTS
# ============================================================

cv2.imwrite(
    "band_pass_result.jpg",
    band_pass_result
)

cv2.imwrite(
    "band_reject_result.jpg",
    band_reject_result
)

# ============================================================
# 8. DISPLAY RESULTS
# ============================================================

plt.figure(figsize=(15, 10))

# Original image
plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Textured Image")
plt.axis("off")

# Frequency spectrum
plt.subplot(2, 3, 2)
plt.imshow(spectrum, cmap="gray")
plt.title("Frequency Spectrum")
plt.axis("off")

# Band-pass mask
plt.subplot(2, 3, 3)
plt.imshow(band_pass, cmap="gray")
plt.title("Band-Pass Filter")
plt.axis("off")

# Band-pass result
plt.subplot(2, 3, 4)
plt.imshow(band_pass_result, cmap="gray")
plt.title("Band-Pass Result")
plt.axis("off")
```

```
# Band-reject mask
plt.subplot(2, 3, 5)
plt.imshow(band_reject, cmap="gray")
plt.title("Band-Reject Filter")
plt.axis("off")

# Band-reject result
plt.subplot(2, 3, 6)
plt.imshow(band_reject_result, cmap="gray")
plt.title("Band-Reject Result")
plt.axis("off")

plt.tight_layout()
plt.show()

# ============================================================
# 9. PRINT INFORMATION
# ============================================================

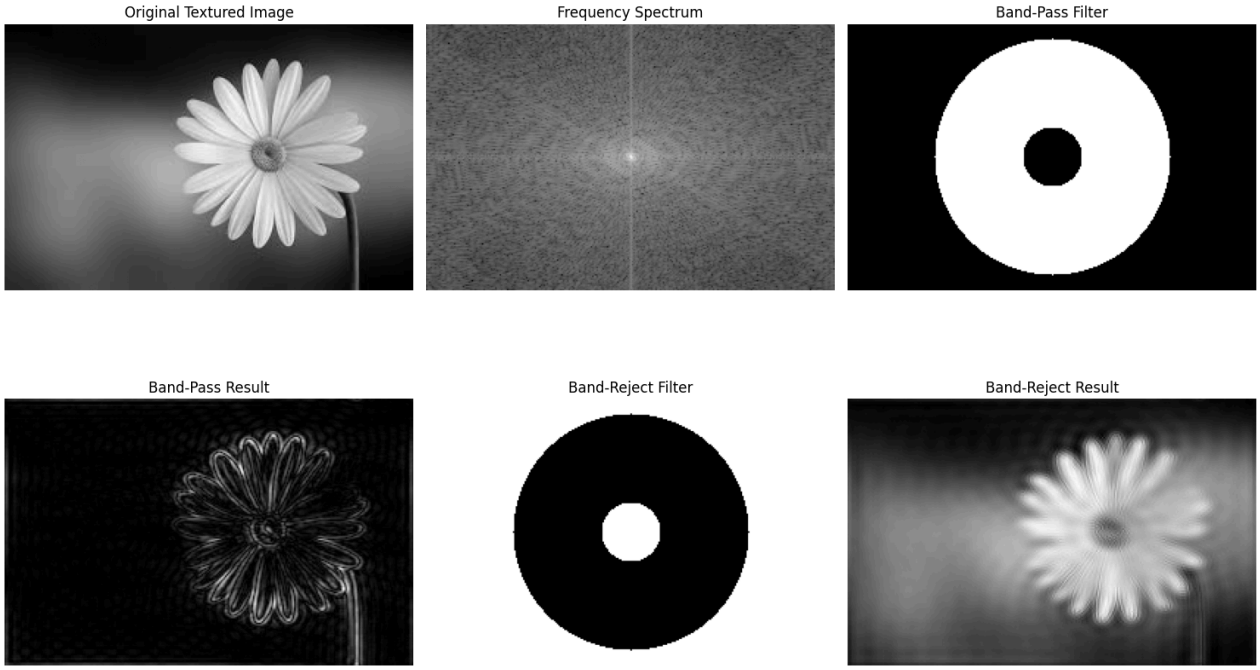
print("=" * 60)
print("BAND-PASS / BAND-REJECT FILTER DESIGNER")
print("=" * 60)

print("Image size       :", image.shape)
print("Low cutoff      :", low_cutoff)
print("High cutoff     :", high_cutoff)

print("\nBand-Pass filtering completed.")
print("Saved as: band_pass_result.jpg")

print("\nBand-Reject filtering completed.")
print("Saved as: band_reject_result.jpg")

print("=" * 60)
```




```
============================================================
BAND-PASS / BAND-REJECT FILTER DESIGNER
============================================================
Image size       : (181, 278)
Low cutoff      : 20
High cutoff     : 80

Band-Pass filtering completed.
Saved as: band_pass_result.jpg

Band-Reject filtering completed.
Saved as: band_reject_result.jpg
============================================================
```

8. Develop a demo that swaps the phase and magnitude components betweem two images to show their relative importance.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt



# ============================================================
# 1. READ TWO IMAGES
# ============================================================

image1 = cv2.imread("/content/images (3).jpg", cv2.IMREAD_GRAYSCALE)
image2 = cv2.imread("/content/images (4).jpg", cv2.IMREAD_GRAYSCALE)

if image1 is None:
    print("Error: image1.jpg not found!")
    exit()
```

```
if image2 is None:
    print("Error: image2.jpg not found!")
    exit()


# ============================================================
# 2. RESIZE IMAGE 2 TO IMAGE 1 SIZE
# ============================================================

image2 = cv2.resize(
    image2,
    (image1.shape[1], image1.shape[0])
)


# ============================================================
# 3. FOURIER TRANSFORM
# ============================================================

fft1 = np.fft.fft2(image1)
fft2 = np.fft.fft2(image2)


# ============================================================
# 4. EXTRACT MAGNITUDE AND PHASE
# ============================================================

magnitude1 = np.abs(fft1)
phase1 = np.angle(fft1)

magnitude2 = np.abs(fft2)
phase2 = np.angle(fft2)


# ============================================================
# 5. SWAP PHASE COMPONENTS
# ============================================================

# Image 1 magnitude + Image 2 phase
phase_swap_1 = magnitude1 * np.exp(1j * phase2)

# Image 2 magnitude + Image 1 phase
phase_swap_2 = magnitude2 * np.exp(1j * phase1)


# ============================================================
# 6. SWAP MAGNITUDE COMPONENTS
# ============================================================

# Image 1 phase + Image 2 magnitude
magnitude_swap_1 = magnitude2 * np.exp(1j * phase1)

# Image 2 phase + Image 1 magnitude
magnitude_swap_2 = magnitude1 * np.exp(1j * phase2)


# ============================================================
# 7. INVERSE FOURIER TRANSFORM
# ============================================================

result_phase_swap_1 = np.fft.ifft2(phase_swap_1)
result_phase_swap_1 = np.abs(result_phase_swap_1)

result_phase_swap_2 = np.fft.ifft2(phase_swap_2)
result_phase_swap_2 = np.abs(result_phase_swap_2)

result_magnitude_swap_1 = np.fft.ifft2(magnitude_swap_1)
result_magnitude_swap_1 = np.abs(result_magnitude_swap_1)

result_magnitude_swap_2 = np.fft.ifft2(magnitude_swap_2)
result_magnitude_swap_2 = np.abs(result_magnitude_swap_2)


# ============================================================
# 8. NORMALIZE RESULTS
# ============================================================

def normalize_image(img):
    return cv2.normalize(
        img,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    ).astype(np.uint8)


result_phase_swap_1 = normalize_image(
    result_phase_swap_1
)

result_phase_swap_2 = normalize_image(
    result_phase_swap_2
)

result_magnitude_swap_1 = normalize_image(
    result_magnitude_swap_1
)

result_magnitude_swap_2 = normalize_image(
    result_magnitude_swap_2
)


# ============================================================
# 9. SAVE OUTPUTS
# ============================================================

cv2.imwrite(
    "phase_swap_image1mag_image2phase.jpg",
    result_phase_swap_1
```

```
)

cv2.imwrite(
    "phase_swap_image2mag_image1phase.jpg",
    result_phase_swap_2
)

cv2.imwrite(
    "magnitude_swap_image1phase_image2mag.jpg",
    result_magnitude_swap_1
)

cv2.imwrite(
    "magnitude_swap_image2phase_image1mag.jpg",
    result_magnitude_swap_2
)


# ============================================================
# 10. DISPLAY RESULTS
# ============================================================

plt.figure(figsize=(16, 12))


# Original Image 1
plt.subplot(3, 3, 1)
plt.imshow(image1, cmap="gray")
plt.title("Image 1")
plt.axis("off")


# Original Image 2
plt.subplot(3, 3, 2)
plt.imshow(image2, cmap="gray")
plt.title("Image 2")
plt.axis("off")


# Magnitude of Image 1
plt.subplot(3, 3, 3)
plt.imshow(
    np.log(magnitude1 + 1),
    cmap="gray"
)
plt.title("Magnitude - Image 1")
plt.axis("off")


# Phase of Image 1
plt.subplot(3, 3, 4)
plt.imshow(phase1, cmap="gray")
plt.title("Phase - Image 1")
plt.axis("off")


# Phase swapped
plt.subplot(3, 3, 5)
plt.imshow(
    result_phase_swap_1,
    cmap="gray"
)
plt.title("Image 1 Magnitude + Image 2 Phase")
plt.axis("off")


# Magnitude of Image 2
plt.subplot(3, 3, 6)
plt.imshow(
    np.log(magnitude2 + 1),
    cmap="gray"
)
plt.title("Magnitude - Image 2")
plt.axis("off")


# Phase of Image 2
plt.subplot(3, 3, 7)
plt.imshow(phase2, cmap="gray")
plt.title("Phase - Image 2")
plt.axis("off")


# Magnitude swapped
plt.subplot(3, 3, 8)
plt.imshow(
    result_magnitude_swap_1,
    cmap="gray"
)
plt.title("Image 2 Magnitude + Image 1 Phase")
plt.axis("off")


# Explanation
plt.subplot(3, 3, 9)
plt.axis("off")

plt.text(
    0.05,
    0.7,
    "Phase contains important\n"
    "structural information.\n\n"
    "Magnitude controls the\n"
    "frequency strength.",
    fontsize=13
)

plt.tight_layout()
plt.show()


=
```

```
# 11. PRINT RESULTS
# ============================================================

print("=" * 65)
print("PHASE AND MAGNITUDE SWAPPING DEMO")
print("=" * 65)

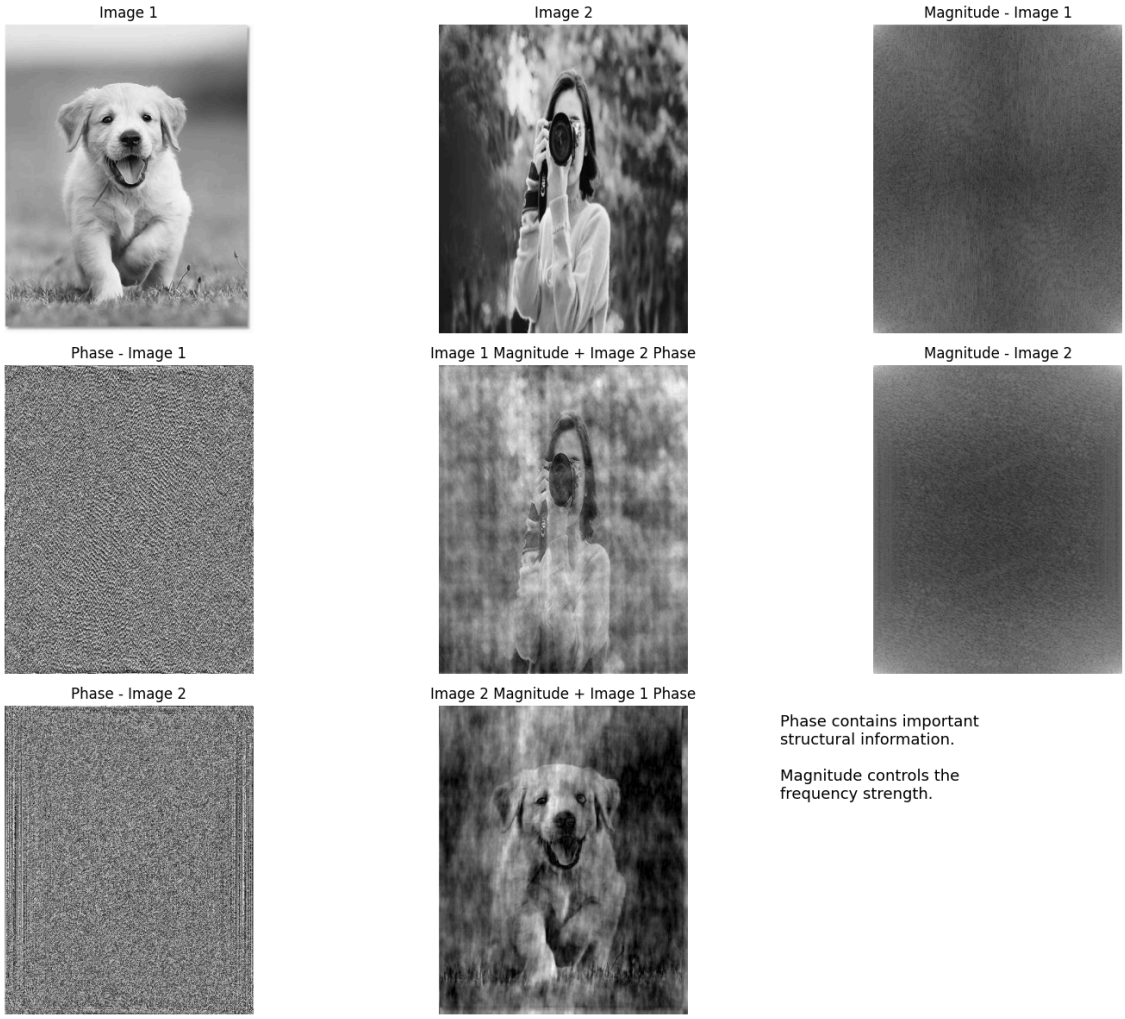
print("\nImage 1 size:", image1.shape)
print("Image 2 size:", image2.shape)

print("\nPhase swapping completed.")
print("Image 1 Magnitude + Image 2 Phase:")
print("phase_swap_image1mag_image2phase.jpg")

print("\nMagnitude swapping completed.")
print("Image 2 Magnitude + Image 1 Phase:")
print("magnitude_swap_image2phase_image1mag.jpg")

print("\nObservation:")
print("Phase carries significant structural and spatial information.")
print("Magnitude represents the strength of different frequencies.")

print("=" * 65)
```




```
=================================================================
PHASE AND MAGNITUDE SWAPPING DEMO
=================================================================

Image 1 size: (503, 406)
Image 2 size: (503, 406)

Phase swapping completed.
Image 1 Magnitude + Image 2 Phase:
phase_swap_image1mag_image2phase.jpg

Magnitude swapping completed.
Image 2 Magnitude + Image 1 Phase:
magnitude_swap_image2phase_image1mag.jpg

Observation:
Phase carries significant structural and spatial information.
Magnitude represents the strength of different frequencies.
=================================================================
```

9. Create a blur defectiopn tool that flags blurry images based on the high frequency energy content of their spectrum.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt


# ============================================================
# 1. READ IMAGE
# ============================================================

image = cv2.imread("/content/images (3).jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: image.jpg not found!")
    exit()


# ============================================================
# 2. FOURIER TRANSFORM
# ============================================================

dft = np.fft.fft2(image)

# Move low frequencies to the center
dft_shift = np.fft.fftshift(dft)


# ============================================================
# 3. CALCULATE MAGNITUDE SPECTRUM
# ============================================================

magnitude = np.abs(dft_shift)

# Log scale for visualization
spectrum = np.log1p(magnitude)


# ============================================================
# 4. CREATE HIGH-FREQUENCY MASK
# ============================================================

rows, cols = image.shape

crow = rows // 2
ccol = cols // 2

Y, X = np.ogrid[:rows, :cols]

distance = np.sqrt(
    (X - ccol) ** 2 +
    (Y - crow) ** 2
)


# Radius of low-frequency region
radius = 30

# High-frequency region
high_frequency_mask = distance > radius


# ============================================================
# 5. CALCULATE HIGH-FREQUENCY ENERGY
# ============================================================

high_frequency_energy = np.sum(
    magnitude[high_frequency_mask] ** 2
)

total_frequency_energy = np.sum(
    magnitude ** 2
)

# Percentage of energy in high frequencies
high_frequency_percentage = (
    high_frequency_energy /
    total_frequency_energy
) * 100


# ============================================================
# 6. BLUR THRESHOLD
# ============================================================

# Change this value according to your images
threshold = 5.0


if high_frequency_percentage < threshold:
    result = "BLURRY IMAGE"
else:
    result = "SHARP IMAGE"


# ============================================================
# 7. EXTRACT HIGH-FREQUENCY SPECTRUM
# ============================================================

high_frequency_spectrum = (
    magnitude * high_frequency_mask
)

high_frequency_spectrum = np.log1p(
    high_frequency_spectrum
)


# ============================================================
# 8. DISPLAY RESULTS
# ============================================================

plt.figure(figsize=(15, 10))
```

```
# Original image
plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Input Image")
plt.axis("off")


# Frequency spectrum
plt.subplot(2, 3, 2)
plt.imshow(spectrum, cmap="gray")
plt.title("FFT Spectrum")
plt.axis("off")


# High frequency mask
plt.subplot(2, 3, 3)
plt.imshow(
    high_frequency_mask,
    cmap="gray"
)
plt.title("High-Frequency Mask")
plt.axis("off")


# High frequency spectrum
plt.subplot(2, 3, 4)
plt.imshow(
    high_frequency_spectrum,
    cmap="gray"
)
plt.title("High-Frequency Spectrum")
plt.axis("off")


# Original image again with result
plt.subplot(2, 3, 5)
plt.imshow(image, cmap="gray")
plt.title(result)
plt.axis("off")


# Information panel
plt.subplot(2, 3, 6)
plt.axis("off")

plt.text(
    0.05,
    0.75,
    "BLUR DETECTION RESULT",
    fontsize=14,
    fontweight="bold"
)

plt.text(
    0.05,
    0.55,
    f"High-Frequency Energy:\n"
    f"{high_frequency_percentage:.2f}%",
    fontsize=12
)

plt.text(
    0.05,
    0.35,
    f"Threshold:\n{threshold:.2f}%",
    fontsize=12
)

plt.text(
    0.05,
    0.15,
    f"Status:\n{result}",
    fontsize=12
)


plt.tight_layout()
plt.show()


# ============================================================
# 9. SAVE OUTPUT SPECTRUM
# ============================================================

spectrum_output = cv2.normalize(
    spectrum,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

cv2.imwrite(
    "frequency_spectrum.jpg",
    spectrum_output
)


# ============================================================
# 10. PRINT RESULT
# ============================================================

print("=" * 60)
print("FREQUENCY DOMAIN BLUR DETECTION")
print("=" * 60)

print(f"Image size                : {image.shape}")
print(f"High-frequency energy     : {high_frequency_percentage:.2f}%")
print(f"Blur threshold            : {threshold:.2f}%")

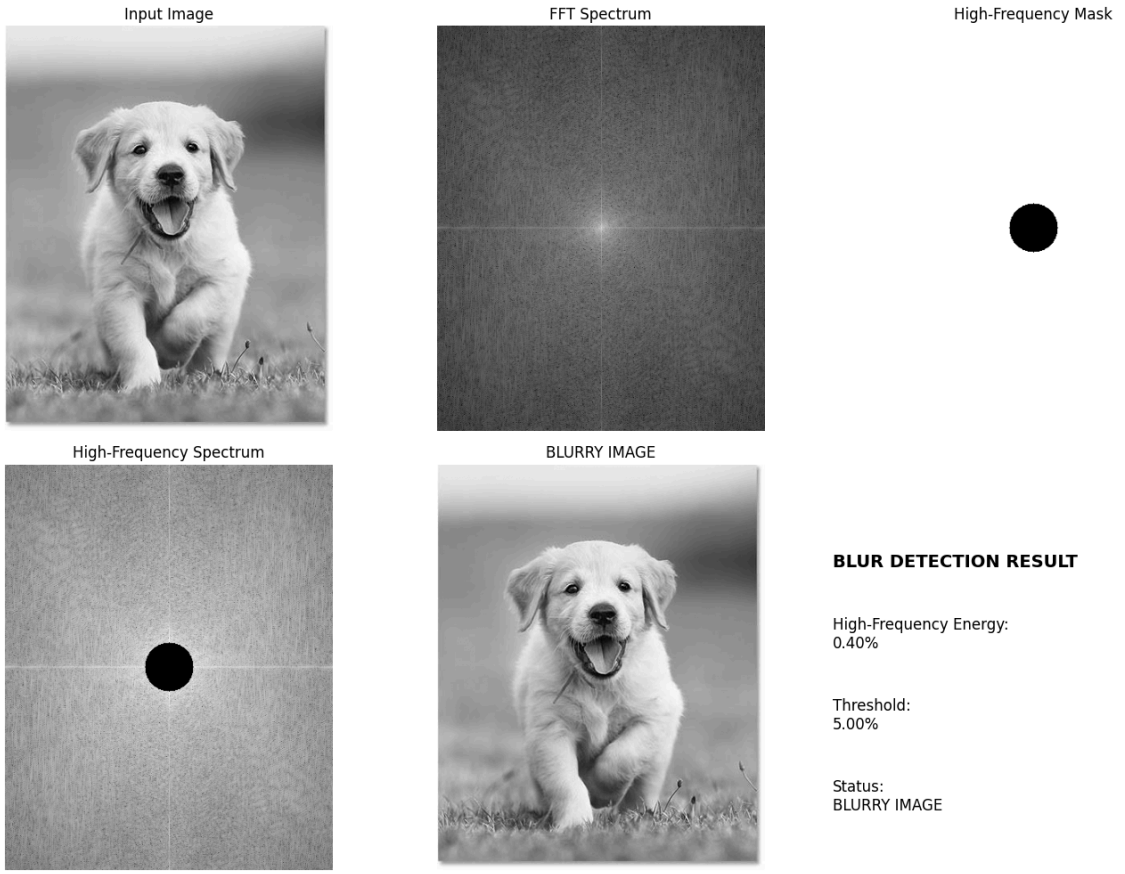
print("\nFinal Result:")
```

```
print(result)

if result == "BLURRY IMAGE":
    print("Reason: Low high-frequency energy detected.")
else:
    print("Reason: Sufficient high-frequency energy detected.")

print("\nFrequency spectrum saved as:")
print("frequency_spectrum.jpg")

print("=" * 60)
```




```
============================================================
FREQUENCY DOMAIN BLUR DETECTION
============================================================
Image size                  : (503, 406)
High-frequency energy       : 0.40%
Blur threshold              : 5.00%

Final Result:
BLURRY IMAGE
Reason: Low high-frequency energy detected.

Frequency spectrum saved as:
frequency_spectrum.jpg
============================================================
```

10. Build a side by side visualizer comparing spatial domain and frquency domain filtering results for the same image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt


# ============================================================
# 1. READ IMAGE
# ============================================================

image = cv2.imread("/content/images (4).jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: image.jpg not found!")
    exit()


# ============================================================
# 2. SPATIAL DOMAIN FILTERING
# ============================================================

# Gaussian Blur in spatial domain
spatial_result = cv2.GaussianBlur(
    image,
    (21, 21),
    0
)
```

```
# ============================================================
# 3. FREQUENCY DOMAIN FILTERING
# ============================================================

# Convert image to float
image_float = np.float32(image)

# Apply 2D Fourier Transform
dft = np.fft.fft2(image_float)

# Shift low frequencies to center
dft_shift = np.fft.fftshift(dft)


# ============================================================
# 4. CREATE GAUSSIAN LOW-PASS FILTER
# ============================================================

rows, cols = image.shape

crow = rows // 2
ccol = cols // 2

Y, X = np.ogrid[:rows, :cols]

# Distance from center
distance_squared = (
    (X - ccol) ** 2 +
    (Y - crow) ** 2
)

# Gaussian cutoff
sigma = 40

gaussian_filter = np.exp(
    -distance_squared /
    (2 * sigma * sigma)
)


# ============================================================
# 5. APPLY FILTER IN FREQUENCY DOMAIN
# ============================================================

filtered_dft = dft_shift * gaussian_filter


# ============================================================
# 6. INVERSE FOURIER TRANSFORM
# ============================================================

# Shift back
filtered_dft = np.fft.ifftshift(filtered_dft)

# Inverse FFT
frequency_result = np.fft.ifft2(filtered_dft)

# Take real part
frequency_result = np.abs(frequency_result)


# Normalize to 0-255
frequency_result = cv2.normalize(
    frequency_result,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)


# ============================================================
# 7. CALCULATE DIFFERENCE
# ============================================================

difference = cv2.absdiff(
    spatial_result,
    frequency_result
)


# ============================================================
# 8. FREQUENCY SPECTRUM
# ============================================================

spectrum = np.log(
    1 + np.abs(dft_shift)
)


# ============================================================
# 9. SAVE OUTPUTS
# ============================================================

cv2.imwrite(
    "spatial_filtered.jpg",
    spatial_result
)

cv2.imwrite(
    "frequency_filtered.jpg",
    frequency_result
)

cv2.imwrite(
    "difference.jpg",
    difference
)
```

```
# ============================================================
# 10. SIDE-BY-SIDE VISUALIZER
# ============================================================

plt.figure(figsize=(16, 10))


# Original image
plt.subplot(2, 3, 1)

plt.imshow(
    image,
    cmap="gray"
)

plt.title("Original Image")
plt.axis("off")


# Spatial domain result
plt.subplot(2, 3, 2)

plt.imshow(
    spatial_result,
    cmap="gray"
)

plt.title(
    "Spatial Domain\nGaussian Blur"
)

plt.axis("off")


# Frequency domain result
plt.subplot(2, 3, 3)

plt.imshow(
    frequency_result,
    cmap="gray"
)

plt.title(
    "Frequency Domain\nGaussian Low-Pass"
)

plt.axis("off")


# Frequency spectrum
plt.subplot(2, 3, 4)

plt.imshow(
    spectrum,
    cmap="gray"
)

plt.title(
    "Original Frequency Spectrum"
)

plt.axis("off")


# Gaussian frequency filter
plt.subplot(2, 3, 5)

plt.imshow(
    gaussian_filter,
    cmap="gray"
)

plt.title(
    "Gaussian Frequency Filter"
)

plt.axis("off")


# Difference
plt.subplot(2, 3, 6)

plt.imshow(
    difference,
    cmap="gray"
)

plt.title(
    "Difference Between Results"
)

plt.axis("off")


plt.tight_layout()
plt.show()


# ============================================================
# 11. PRINT INFORMATION
# ============================================================

print("=" * 65)
print("SPATIAL vs FREQUENCY DOMAIN FILTERING")
print("=" * 65)

print("Image size:", image.shape)

print("\nSpatial Domain:")
print("Gaussian Blur kernel = 21 x 21")

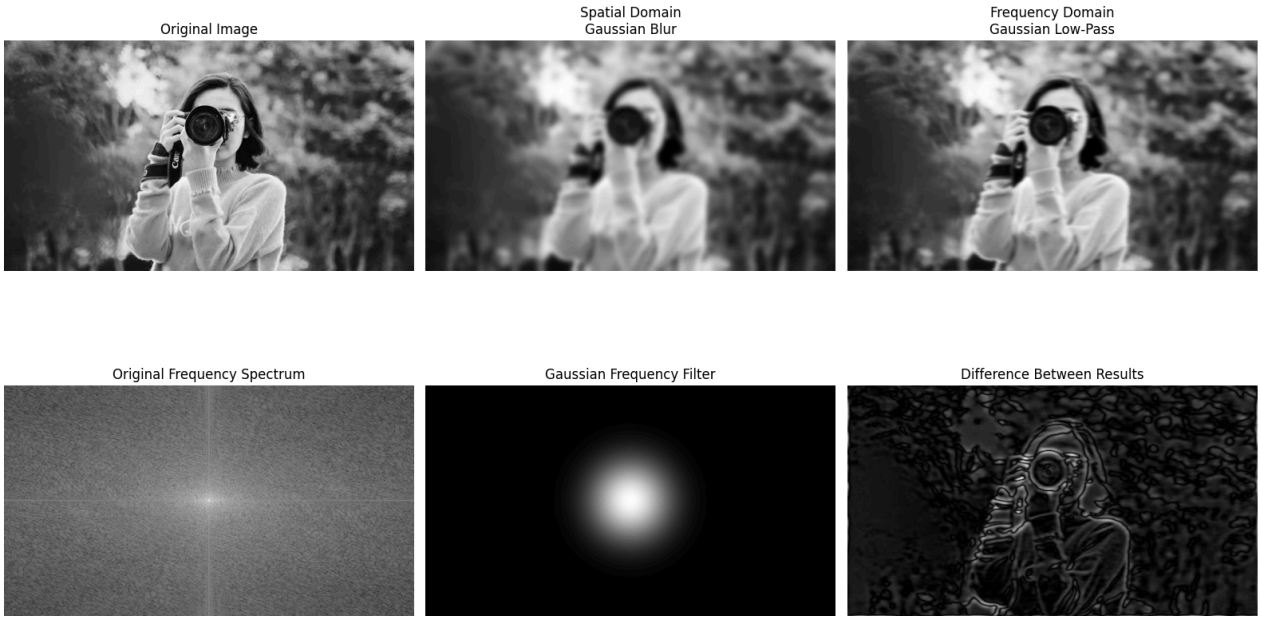
print("\nFrequency Domain:")
```

```
print("Gaussian Low-Pass Filter")
print("Sigma =", sigma)

print("\nOutput files:")
print("1. spatial_filtered.jpg")
print("2. frequency_filtered.jpg")
print("3. difference.jpg")

print("\nComparison completed successfully.")

print("=" * 65)
```




```
=================================================================
SPATIAL vs FREQUENCY DOMAIN FILTERING
=================================================================
Image size: (335, 597)

Spatial Domain:
Gaussian Blur kernel = 21 x 21

Frequency Domain:
Gaussian Low-Pass Filter
Sigma = 40

Output files:
1. spatial_filtered.jpg
2. frequency_filtered.jpg
3. difference.jpg

Comparison completed successfully.
=================================================================
```